EXPERIMENT 5
COMPARATIVE ANALYSIS OF VGG, RESNET, AND GOOGLENET ON IMAGE CLASSIFICATION

Problem Statement

Implement and compare VGG, ResNet, and GoogLeNet CNN architectures for image classification and evaluate their performance.

Suggested Dataset

The lab manual suggests Dogs vs. Cats. This runnable implementation uses CIFAR-10 from torchvision for convenient loading and compares all 10 classes; therefore it is a practical substitute dataset.

Objectives

1. Understand the architectural differences between VGG, ResNet, and GoogLeNet.
2. Use pre-trained CNN models with adapted output layers.
3. Train and evaluate the models under the same dataset conditions.
4. Compare test accuracy using a chart.

Tools and Libraries Used

Python 3.x, PyTorch, torchvision, Matplotlib

Procedure

5. Import libraries and select CPU/GPU.
6. Load and normalize CIFAR-10 images.
7. Load pre-trained VGG16, ResNet18, and GoogLeNet.
8. Replace the final classification layers for 10 classes.
9. Train each model and calculate test accuracy.
10. Plot and compare the measured accuracies.

Step 1: Import Libraries and Configure Device

```
import torch
import torch.nn as nn
import torch.optim as optim
from torchvision import datasets, models, transforms
from torch.utils.data import DataLoader, Subset
import matplotlib.pyplot as plt

device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
print("Using device:", device)
```

Result: Required libraries are imported and the available device is selected.

Step 2: Load and Preprocess Dataset

```
transform = transforms.Compose([
    transforms.Resize((96, 96)),
    transforms.ToTensor(),
    transforms.Normalize((0.5, 0.5, 0.5), (0.5, 0.5, 0.5))
```

```
])

train_full = datasets.CIFAR10(root="data", train=True,
                              download=True, transform=transform)
test_full = datasets.CIFAR10(root="data", train=False,
                             download=True, transform=transform)

trainset = Subset(train_full, range(5000))
testset = Subset(test_full, range(1000))
trainloader = DataLoader(trainset, batch_size=32, shuffle=True)
testloader = DataLoader(testset, batch_size=32, shuffle=False)

print("Training images:", len(trainset))
print("Testing images:", len(testset))
```

Result: A subset of CIFAR-10 is resized, normalized, and loaded in batches to reduce execution time.

Step 3: Load and Configure the Architectures

```
vgg = models.vgg16(weights=models.VGG16_Weights.DEFAULT)
vgg.classifier[6] = nn.Linear(vgg.classifier[6].in_features, 10)

resnet = models.resnet18(weights=models.ResNet18_Weights.DEFAULT)
resnet.fc = nn.Linear(resnet.fc.in_features, 10)

googlenet = models.googlenet(
    weights=models.GoogLeNet_Weights.DEFAULT, aux_logits=True)
googlenet.fc = nn.Linear(googlenet.fc.in_features, 10)
googlenet.aux1.fc2 = nn.Linear(googlenet.aux1.fc2.in_features, 10)
googlenet.aux2.fc2 = nn.Linear(googlenet.aux2.fc2.in_features, 10)

models_dict = {
    "VGG16": vgg.to(device),
    "ResNet18": resnet.to(device),
    "GoogLeNet": googlenet.to(device)
}
print("All three models are ready.")
```

Result: Pre-trained models are loaded and their classification heads are adapted to produce 10 class scores.

Step 4: Train and Evaluate Each Model

```
def train_evaluate(model, epochs=2):
    criterion = nn.CrossEntropyLoss()
    optimizer = optim.Adam(model.parameters(), lr=0.0001)

    for epoch in range(epochs):
        model.train()
        total_loss = 0.0
        for images, labels in trainloader:
            images, labels = images.to(device), labels.to(device)
            optimizer.zero_grad()
            outputs = model(images)
            if isinstance(outputs, tuple):
                outputs = outputs[0]
            loss = criterion(outputs, labels)
            loss.backward()
            optimizer.step()
            total_loss += loss.item()
```

```
        print(f"Epoch {epoch+1}/{epochs}, Loss: "
              f"{total_loss / len(trainloader):.4f}")

    model.eval()
    correct, total = 0, 0
    with torch.no_grad():
        for images, labels in testloader:
            images, labels = images.to(device), labels.to(device)
            outputs = model(images)
            if isinstance(outputs, tuple):
                outputs = outputs[0]
            predicted = outputs.argmax(dim=1)
            total += labels.size(0)
            correct += (predicted == labels).sum().item()
    return 100 * correct / total

accuracies = {}
for name, model in models_dict.items():
    print("\nTraining", name)
    accuracies[name] = train_evaluate(model)
    print(f"{name} Test Accuracy: {accuracies[name]:.2f}%")
```

Result: Each architecture is trained and evaluated, and its test accuracy is printed.

Step 5: Plot Accuracy Comparison

```
plt.figure(figsize=(7, 4))
plt.bar(accuracies.keys(), accuracies.values())
plt.ylabel("Test Accuracy (%)")
plt.title("VGG16 vs ResNet18 vs GoogLeNet")
plt.ylim(0, 100)
plt.tight_layout()
plt.show()
print("Accuracy comparison:", accuracies)
```

Output

```
Using device: cuda
Training images: 5000
Testing images: 1000

Training VGG16
Epoch 1/2, Loss: <measured loss>
Epoch 2/2, Loss: <measured loss>
VGG16 Test Accuracy: <measured accuracy>%

Training ResNet18
Epoch 1/2, Loss: <measured loss>
Epoch 2/2, Loss: <measured loss>
ResNet18 Test Accuracy: <measured accuracy>%

Training GoogLeNet
Epoch 1/2, Loss: <measured loss>
Epoch 2/2, Loss: <measured loss>
GoogLeNet Test Accuracy: <measured accuracy>%
```

Comparison

Architecture	Main Feature	General Observation
VGG16	Stacked 3×3 convolution layers	Deep design; relatively high memory and computation.
ResNet18	Residual/skip connections	Helps information and gradients flow through layers.
GoogLeNet	Inception modules	Extracts features at multiple scales efficiently.

Conclusion

This experiment compares VGG16, ResNet18, and GoogLeNet using transfer learning for image classification. VGG uses stacked convolution layers, ResNet uses residual connections, and GoogLeNet uses Inception modules. The model with the best measured accuracy can be selected while also considering computational cost.